

Deep Research — REST POST /api/v1/ translate target-language fix

Revision: 1 **Last modified:** 2026-06-15T00:00:00Z **Authority:** §11.4.150 deep multi-angle web research (run in parallel with the fix per §11.4.150(F)) **Scope:** Fix (3) — the REST handler hardcoded RU→SR; the request struct had no `target_lang/source_lang`. The fix adds those fields, threads them through the pipeline, and preserves the previous RU→SR default for back-compat.

The question

1. Is adding `target_lang/source_lang` to the JSON request body (with a preserved default) the best REST design for translation language selection, or should it use `Accept-Language` / query params? (ii) Are we masking a deeper problem — specifically, **is adding `target_lang` a backward-compatibility break** for existing callers, and is hardcoding any single default the right fallback?

Angle 1 — REST i18n / language-selection design conventions (external authority)

Authoritative i18n REST guidance recognizes three placements for the requested language: the `Accept-Language` header (RFC 5646), a language query parameter, and a request-body field. For a **translation API specifically** — where the target language is the core *payload semantics* of the operation, not a presentation/locale preference for the response wrapper — the language belongs in the **request body** alongside the text being translated. Key distinction:

- `Accept-Language` and locale query params describe “what language do you want the *response framing* in” (error messages, metadata) — a presentation concern.
- A translation request’s source/target language is the operation’s primary input — it determines *what work is done*, not how the result is framed. Putting it in the body (`source_lang/target_lang`) matches how translation services model it (e.g. Google Translate v2’s `target/source` are call parameters, not `Accept-Language`).

So body fields are the correct placement for a translate endpoint. Naming `source_lang/target_lang` is clear and conventional (BCP-47/RFC 5646 tags should be accepted as values).

- STAC API Language (I18N) extension — language placement, RFC 5646 tags, Vary for caches — <https://github.com/stac-api-extensions/language/blob/main/README.md> (accessed 2026-06-15)

- Google Cloud Translate client — target/source as call parameters, target optional with a default — https://docs.cloud.google.com/python/docs/reference/translate/latest/google.cloud.translate_v2.client.Client.html (accessed 2026-06-15)
- Spring Boot REST i18n example (Accept-Language is for response localization, distinct concern) — <https://howtodoinjava.com/spring-boot/rest-i18n-example/> (accessed 2026-06-15)

Angle 2 — Backward compatibility of adding optional body fields + preserved default (no-deeper-problem check, §11.4.150(C))

The core back-compat worry: does adding `target_lang/source_lang` break existing callers who don't send them?

No — additive optional fields are a non-breaking change, on two independent grounds: 1. **JSON deserialization tolerates absent fields**. An existing client that POSTs the old body (without the new fields) deserializes fine; the new fields take their zero/empty value. This is the standard “additive, optional $\Rightarrow$ compatible” API-evolution rule. 2. **The fix preserves the previous default**. When `target_lang/source_lang` are empty, the handler falls back to the historical RU $\rightarrow$ SR behaviour. So a pre-fix caller gets byte-identical behaviour to before the fix — the exact definition of backward compatibility. New callers gain the ability to choose languages.

This matches the client-library convention that “target language can be optional with a default ... supporting backward compatibility when the parameter is not explicitly provided.”

Deeper-problem check (§11.4.150(C)): the genuine smell to avoid is a hardcoded default that *silently mistranslates* for callers who assumed a different pair. Mitigations to confirm in the fix: - The preserved RU $\rightarrow$ SR default must be **documented** as the no-language default (so it is a contract, not a surprise). Recommended (non-blocking): once clients are migrated, consider returning a 400 on an *unknown/unsupported* explicit language pair rather than silently coercing — but an *absent* pair correctly uses the documented default for back-compat. - Source-language: if `source_lang` is empty, confirm whether the pipeline can auto-detect or must assume RU; an unstated assumption here is the residual risk. State it explicitly in the API docs. - Values should be validated against supported BCP-47/RFC 5646 tags; an invalid explicit tag should error, not fall back to the default (falling back would mask a client bug).

Forward-looking, optionally accept Accept-Language as a *secondary* signal for response framing only — but body fields remain authoritative for the translation operation, per Angle 1.

- API evolution — additive optional fields are non-breaking; preserved default = compatible (STAC Vary/older-client guidance) — <https://github.com/stac-api-extensions/language/blob/main/README.md> (accessed 2026-06-15)
- Google Translate — optional target with default = back-compat — https://docs.cloud.google.com/python/docs/reference/translate/latest/google.cloud.translate_v2.client.Client.html (accessed 2026-06-15)

Finding

Best-practice-confirmed. Putting `source_lang/target_lang` in the `translate` request body is the correct REST design for a translation endpoint (language is operation input, not response-locale framing), and adding them as **optional fields with the preserved RU→SR default is a non-breaking, backward-compatible change** — existing callers get identical behaviour, new callers gain language choice. No deeper problem masked. Non-blocking improvements: (a) document the RU→SR no-language default as an explicit contract; (b) validate explicit tags against supported BCP-47 values and error (don't silently fall back) on an unknown explicit pair; (c) document the empty-`source_lang` behaviour (auto-detect vs assume-RU).

Deep-research 2026-06-15: <https://github.com/stac-api-extensions/language/blob/main/README.md>, https://docs.cloud.google.com/python/docs/reference/translate/latest/google.cloud.translate_v2.client.Client.html, <https://howtodoinjava.com/spring-boot/rest-api-example/>